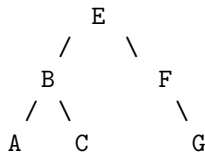


Assignment 4: Morse Scrambled Cipher



Description

For this assignment, you will need to implement a Morse-Scrambler Cipher Algorithm using a Binary Search Tree (BST), to encrypt/decrypt a file. Each character will need to be mapped to a Morse code for encryption and for decryption you will need to map a Morse code to a letter. You construct a Morse code by inserting letters one by one into a BST and record the path taken to find the letter. For example given the following BST:



The path to C starting from the root is left and then right, thus the morse code for C would be LR. You will need to implement the following binary search tree class:

```

#include <cstdlib>
#include <string>

#ifndef BST_H
#define BST_H

template <class t1, class t2>
class bst
{
private:
    class binTreeNode
    {
    public:
        binTreeNode() : key( t1() ), value( t2() ), left(nullptr), right(nullptr) {}
        binTreeNode(t1 k, t2 v) : key(k), value(v), left(nullptr), right(nullptr) {}

        t1 key;

```

```

    t2 value;

    binTreeNode * left;
    binTreeNode * right;
};

binTreeNode * root;

void deallocateTree(binTreeNode*);

binTreeNode* insert(binTreeNode*, t1, t2);
void update(binTreeNode*, t1, t2);
t2 getValue(binTreeNode*, t1);

public:
    bst() : root(nullptr) {}
    ~bst() { deallocateTree(root); }
    void insert(t1 k, t2 v) { root = insert(root, k, v); }
    void update(t1 k, t2 v) { update(root, k, v); }
    t2 getValue(t1 k) { return getValue(root, k); }
    std::string getPath(t1);
};

#endif

```

Each member of the binary tree class contains/performs the following

- `class binTreeNode` is a node within the binary search class, each node contains a key, value pair along with left and right children pointers, the constructors are implemented for you
- `binTreeNode * root` - is a the root node for the binary search tree
- `bst<t1, t2>::bst()` - default constructor (implemented for you)
- `bst<t1, t2>::~~bst()` - destructor (implemented for you)
- `void bst<t1, t2>::deallocateTree(binTreeNode * r)` - recursive function that deallocates the binary search tree in postorder fashion
- `void bst<t1, t2>::insert(t1 k, t2 v)` - function that inserts a new key, value pair into the binary search tree (implemented for you)
- `typename bst<t1, t2>::binTreeNode* bst<t1, t2>::insert(binTreeNode * r, t1 k, t2 v)` - recursive binary search tree function that finds the location of where the new key would go and insert a new node in this vacancy in the tree
- `void bst<t1, t2>::update(t1 k, t2 v)` - a function that updates a the key in the binary search tree with a new value (implemented for you)
- `void bst<t1, t2>::update(binTreeNode * r, t1 k, t2 v)` - recursive binary search tree function that finds a node with the matching key and reassigns its value
- `t2 bst<t1, t2>::getValue(t1 k)` - a function that returns the value of a node with a matching key (implemented for you)
- `t2 bst<t1, t2>::getValue(binTreeNode * r, t1 k)` - recursive function that finds a node with a matching key and returns the value

- `std::string bst<t1, t2>::getPath(t1 k)` - a function that returns the left/right path from the root node to the node with the matching key, returns "S" if the root node matches the parameter k (this will not be a recursive function)

Contents of Main (Encryption/Decryption)

In your main, you first have to read the following using standard input `cin`

```
lettersFileName TextToEncryptOrDecryptFileName Choice
```

The input will be space separated. The letters file contains a set of characters (one per line), it will contain all the letters 'A' through 'Z' (upper and lower case), '0' through '9', also contains dot '.', ',', and space character and end of line character, thus you will need to use either `getline` function or `cin.get` since you will need to read the whitespace characters as well. The characters in the letters file are in any order.

The text to encrypt/decrypt file contains either regular text that you will need to convert into morse code or the text is already in morse code and needs to be decrypted.

The choice character will be 'E'/'e' which denotes you need to encrypt the text file or 'D'/'d' which denotes to decrypt the text file.

You will then need to read in each character from the letters file and insert each character into a `bst<char, string>` object one at a time. Once all inserted, you then need to update each character/key value field to its correct path (the path/value will be each characters morse code). Then you will need to create another object `bst<string, char>` which maps each morse code to the original letter. So you will need to find a way to insert each element from `bst<char, string>` object into the `bst<string, char>` object (essentially reversing `letter=>morse` to `morse=>letter`).

If the choice is to encrypt, you read each character from the `TextToEncrypt` file and for each character, output the morse code to standard output `cout` and sue '#' as a separator for each morse code.

If the choice is to decrypt, you read each string between the '#' and output the original letter one by one to standard output `cout`.

Sample Run

```
% g++ bst.cpp main.cpp
% ./a.out
```

```
letters01.txt re3.txt e
```

```
//Output is one long line full of L/R separated
//by #, the output files are provided
```

Criteria

- Document your code
- No STL allowed
- Must use your `bst<char, string>` and `bst<string, char>` objects for all of your letter to Morse or Morse to letter look ups

Submission

Submit `bst.cpp` and `main.cpp` to code grade by the deadline

References

- Supplemental Video Link [Supplemental Video](#)